

Frida Hooking и обход SSL Pinning для приложения Flutter (Android)

ОПУБЛИКОВАНО:

Красная команда

ОПУБЛИКОВАНО:

28 апреля 2022 г.

ОПУБЛИКОВАНО В:

Технический обмен

TL;DR

Позвольте мне начать этот пост с мема, который кратко описывает мои чувства за последний месяц. Поскольку обход SSL-закрепления для приложения Flutter занимает слишком много времени, я все еще продолжаю исследования и пишу блог в конце месяца :v



В истории моего блога не было статей о пентестинге мобильных приложений, поэтому сегодня я хотел бы взять на себя смелость написать несколько строк знаний об этой области.

В большинстве проектов, которые я выполнял раньше, я сталкивался с мобильными приложениями, проверяющими наличие root-доступа/джейлбрейка, SSL-закрепления и всевозможных других проверок. На данный момент есть два способа решить эту проблему: использовать подключение к Frida или пропатчить мобильное приложение. В зависимости от каждого человека, будут разные варианты выбора. Я выберу тот метод, который требует меньше усилий и времени :v.

Сначала я намеревался писать только о теории, чтобы в следующий раз у меня был повод написать вторую часть (теоретико-прикладную), но, подумав еще раз, понял, что писать блог, в котором говорится только о теории, скучно, поэтому я провел больше вечеров, написав прикладную часть знаний по обходу SSL-закрепления для приложения Flutter, чтобы быть немного более вдумчивым.

В этом блоге я рассказал об основах подключения Frida. Если эта часть вас устраивает, то переходите сразу к **обходу SSL-закрепления в приложении Flutter**, чтобы сэкономить время.

- Представляем Фриду

Динамический инструмент для разработчиков, специалистов по обратной инженерии и исследователей безопасности. Если вы хотите узнать больше, перейдите сюда. (<https://frida.re/>)

У Фриды есть некоторые особенности, показанные ниже.



Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers.

Scriptable

Inject your own scripts into black box processes. Hook any function, spy on crypto APIs or trace private application code, no source code needed. Edit, hit save, and instantly see the results. All without compilation steps or program restarts.

Portable

Works on Windows, macOS, GNU/Linux, iOS, Android, and QNX. Install the Node.js bindings from [npm](#), grab a Python package from [PyPI](#), or use Frida through its [Swift bindings](#), [.NET bindings](#), [Qt/Qml bindings](#), or [C API](#). We also have a scalable [footprint](#).

Free

Frida is and will always be [free software](#) (free as in freedom). We want to empower the next generation of developer tools, and help other free software developers achieve interoperability through reverse engineering.

Battle-tested

We are proud that [NowSecure](#) is using Frida to do fast, deep analysis of mobile apps [at scale](#). Frida has a comprehensive test-suite and has gone through years of rigorous testing across a broad range of use-cases.

Как следует из названия блога, мы поговорим с Фридой о привязке к Android. Так что же такое привязка? → мы предоставляем Frida **API Javascript** (<https://frida.re/docs/javascript-api/>) для прямого взаимодействия с функциями Java, для выполнения таких операций, как: извлечение данных или изменение потока операций приложения по назначению.

- [Запрос в блоге](#)

В этом блоге мне понадобятся следующие инструменты:

- Декомпилятор Java (JD-GUI, Jadx, ...)
- Android-устройство/виртуальная машина (Genymotion, Nox Player, ..)
- Набор инструментов для динамического приборостроения (Frida)
- Гидра или ида
- Apktool.jar
- uber-apk-signer.jar

Установить Frida довольно просто:

```
pip install frida-tools
pip install frida
```

Далее загрузите и установите frida-server на свое устройство Android/виртуальную машину здесь и установите его в соответствии со (<https://github.com/frida/frida/releases>) следующими (<https://frida.re/docs/android/>) инструкциями (<https://frida.re/docs/android/>). Или, если вы хотите, чтобы все работало быстрее, установите Frida Server прямо из Playstore :v Используйте следующую команду, чтобы убедиться, что устройство Android/виртуальная машина подключены к компьютеру.

```
устройство adb -l
```

```
# adb devices -l
* daemon not running; starting now at tcp:5037
* daemon started successfully
List of devices attached
103c84ce                device usb:1-1 product:ginkgo model:Redmi_Note_8 device:ginkgo transport_id:1
```

Далее проверьте, работает ли это:

```
frida-ps -U #Список пакетов и процессов
```

- Основы Фриды Хукинг

Вам необходимо ознакомиться с концепцией жизненного цикла активности, (<https://developer.android.com/guide/components/activities/activity-lifecycle#alc>) чтобы понять, как работает приложение. Đầu tiên, kiểu cú pháp bắt buộc để cấp cho Frida, lưu ý rằng **JavaScript API** (<https://frida.re/docs/javascript-api/>) sẽ được viết ở trong *//do something*

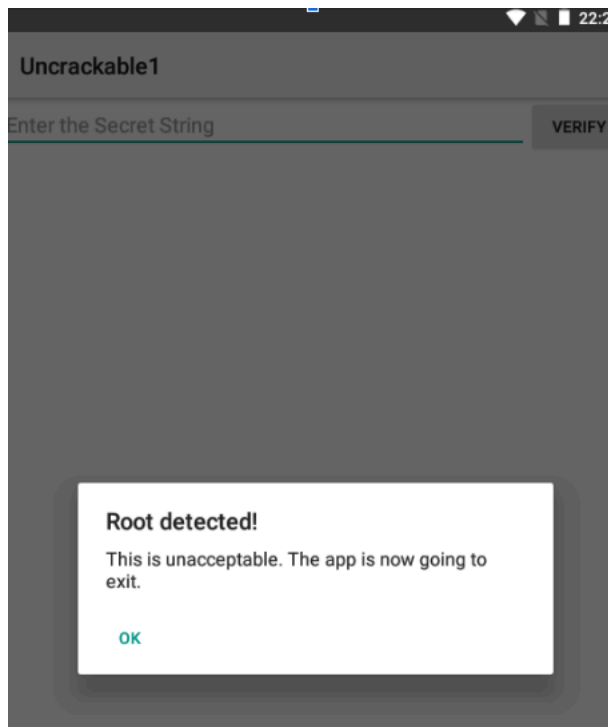


Trong document của Frida thì có nói như sau

- **Java.perform(fn)**: ensure that the current thread is attached to the VM and call **fn**. (This isn't necessary in callbacks from Java.) Will defer calling **fn** if the app's class loader is not available yet. Use **Java.performNow()** if access to the app's classes is not needed.

```
Java.perform(() => {  
    const Activity = Java.use('android.app.Activity');  
    Activity.onResume.implementation = function () {  
        send('onResume() got called! Let\'s call the original implementation');  
        this.onResume();  
    };  
});
```

Bây giờ mình sẽ giải thích một số **JavaScript API** (<https://frida.re/docs/javascript-api/>) cơ bản kèm theo hook một mobile application của OWASP (https://github.com/OWASP/owasp-mstg/blob/master/Crackmes/Android/Level_01/UnCrackable-Level1.apk) nhé (học đi đôi với hành mà :v)
Hình ảnh đầu tiên mở lên thì là dòng chữ **Root detected!** to tướng. Lúc này ứng dụng không thể sử dụng được, nhiệm vụ bây giờ là bypass check root.



Bước đầu tiên là mình cần phải biết được ứng dụng đang check root như thế nào rồi mới hook đúng không, vậy thì dùng **Jadx – Java decompiler** (hoặc dùng bất kỳ tool nào bạn cảm thấy thuận lợi là được) để đọc được java code. Okay, sau khi review thì đây là đoạn code check root

```

11 import sg.vantagepoint.a.c;
12
13 /* Loaded from: classes.dex */
14 public class MainActivity extends Activity {
15     private void a(String str) {
16         AlertDialog create = new AlertDialog.Builder(this).create();
17         create.setTitle(str);
18         create.setMessage("This is unacceptable. The app is now going to exit.");
19         create.setButton(-3, "OK", new DialogInterface.OnClickListener() { // from cl
20             @Override // android.content.DialogInterface.OnClickListener
21             public void onClick(DialogInterface dialogInterface, int i) {
22                 System.exit(0);
23             }
24         });
25         create.setCancelable(false);
26         create.show();
27     }
28
29     @Override // android.app.Activity
30     protected void onCreate(Bundle bundle) {
31         if (c.a() || c.b() || c.c()) {
32             a("Root detected!");
33         }
34         if (b.a(getApplicationContext())) {
35             a("App is debuggable!");
36         }
37         super.onCreate(bundle);
38         setContentView(R.layout.activity_main);
39     }
40 }

```

Class sg.vantagepoint.a.c

```

1 package sg.vantagepoint.a;
2
3 import android.os.Build;
4 import java.io.File;
5
6 /* Loaded from: classes.dex */
7 public class c {
8     public static boolean a() {
9         for (String str : System.getenv("PATH").split(":")) {
10             if (new File(str, "su").exists()) {
11                 return true;
12             }
13         }
14         return false;
15     }
16
17     public static boolean b() {
18         String str = Build.TAGS;
19         return str != null && str.contains("test-keys");
20     }
21
22     public static boolean c() {
23         for (String str : new String[]{"/system/app/Superuser.apk", "/system/xbin/daemonsu", "/sys
24             if (new File(str).exists()) {
25                 return true;
26             }
27         }
28         return false;
29     }
30 }

```

Việc check root của ứng dụng hoạt động bằng cách gọi 3 function a(), b(), c() từ class sg.vantagepoint.a.c

Chỉ cần một trong ba function trên trả về true thì sẽ gọi đến function a(str) để alert thông báo và thoát ứng dụng khi người dùng click vào OK button

Sau khi đã hiểu rõ chương trình check root ra sao thì bước tiếp theo là lên ý tưởng bypass, bằng cách hook vào 3 hàm a(), b(), c() của class sg.vantagepoint.a.c trả về false.

Đoạn code bypass check root khá là đơn giản như sau:



```
//frida -U --no-pause -l bypass_root.js -f owasp.mstg.uncrackable1
Java.perform(function () {
  at('ot', J_vaa, e1, 'sg.vantagepoint.a.c');
  root.a = function () {
    console.log('vao ham a');
    return false;
  };
  root.b.implementation = function () {
    console.log('vao ham b');
    return false;
  };
  root.c.implementation = function () {
    console.log('vao ham c');
    return false;
  };
});
```

Trong đó:

- `sg.vantagepoint.a.c` là class name.
- `Java.use("class")` cho phép **custom method** trong class bằng cách gọi đến **implementation**

Muốn hook và custom method `a()` thuộc class `sg.vantagepoint.a.c` thì dùng cú pháp:

```
root.a.implementation = function () {
  //do something
};
```

- `Java.use(className)`: dynamically get a JavaScript wrapper for `className` that you can instantiate objects from by calling `$new()` on it to invoke a constructor. Call `$dispose()` on an instance to clean it up explicitly (or wait for the JavaScript object to get garbage-collected, or script to get unloaded). Static and non-static methods are available, and you can even replace a method implementation and throw an exception from it:

```
Java.perform(() => {
  const Activity = Java.use('android.app.Activity');
  const Exception = Java.use('java.lang.Exception');
  Activity.onResume.implementation = function () {
    throw Exception.$new('Oh noes!');
  };
});
```

Uses the app's class loader by default, but you may customize this by assigning a different loader instance to

`Java.classFactory.Loader`.

Note that all method wrappers provide a `clone(options)` API to create a new method wrapper with custom NativeFunction options.

Ngoài ra, các bạn có thấy mình có đưa `console.log()`; vào trong code bypass không? Đó là một tip nhỏ mình thường dùng để debug biết được là đoạn liệu code đó có hoạt động?. Nếu các bạn code ngon thì khỏi cần thêm cũng được :v

```
PS C:\Users\NoName\Desktop\blog_frida> frida -U --no-pause -l .\bypass_root.js -f owasp.mstg.uncrackable1

=====
|  _ _ |   Frida 15.0.2 - A world-class dynamic instrumentation toolkit
| ( _ ) |
|  _ _ |   Commands:
|/_/_|_||   help      -> Displays the help system
|  _ _ |   object?    -> Display information about 'object'
|  _ _ |   exit/quit  -> Exit
|  _ _ |
|  _ _ |   More info at https://frida.re/docs/home/
Spawned 'owasp.mstg.uncrackable1'. Resuming main thread!
[SM-G965N::owasp.mstg.uncrackable1]-> vao ham a
vao ham b
vao ham c
```

Okay, sau thử thách bypass check root thành công sẽ là thử thách nhập vào đúng secret string của ứng dụng :v
Tiếp tục công việc đọc hiểu đoạn code xử lý chỗ này để giải quyết thôi



```

41 public void verify(View view) {
42     String str;
43     String obj = ((EditText) findViewById(R.id.edit_text)).getText().toString();
44     AlertDialog create = new AlertDialog.Builder(this).create();
45     if (a.a(obj)) {
46         create.setTitle("Success!");
47         str = "This is the correct secret.";
48     } else {
49         create.setTitle("Nope...");
50         str = "That's not it. Try again.";
51     }
52     create.setMessage(str);
53     create.setButton(-3, "OK", new DialogInterface.OnClickListener() { // from cl
54         @Override // android.content.DialogInterface.OnClickListener
55         public void onClick(DialogInterface dialogInterface, int i) {
56             dialogInterface.dismiss();
57         }
58     });
59     create.show();
60 }

```

(1): Ứng dụng nhận chuỗi ký tự từ người dùng và lưu vào biến *obj*

(2): Truy cập biến *obj* vào method *a()* (của class *a* nằm chung package với class *MainActivity*) để kiểm tra, nếu kết quả của method *a()* về là *true* thì ứng dụng sẽ alert chuỗi *Success! This is the correct secret.*

```

7 public class a {
8     public static boolean a(String str) {
9         byte[] bArr = new byte[0];
10        try {
11            bArr = sg.vantagepoint.a.a.a(b("8d127684cbc37c17616d806cf50473cc"), Base64.decode("5UJiFctbmgbdLXmpl12mkno8HT4Lv8dlat8FxR2G0c=", 0));
12        } catch (Exception e) {
13            Log.d("CodeCheck", "AES error:" + e.getMessage());
14        }
15        return str.equals(new String(bArr));
16    }
17
18    public static byte[] b(String str) {
19        int length = str.length();
20        byte[] bArr = new byte[length / 2];
21        for (int i = 0; i < length; i += 2) {
22            bArr[i / 2] = (byte) ((Character.digit(str.charAt(i), 16) << 4) + Character.digit(str.charAt(i + 1), 16));
23        }
24        return bArr;
25    }
26 }

```

```

1 package sg.vantagepoint.a;
2
3 import javax.crypto.Cipher;
4 import javax.crypto.spec.SecretKeySpec;
5
6 /* Loaded from: classes.dex */
7 public class a {
8     public static byte[] a(byte[] bArr, byte[] bArr2) {
9         SecretKeySpec secretKeySpec = new SecretKeySpec(bArr, "AES/ECB/PKCS7Padding");
10        Cipher instance = Cipher.getInstance("AES");
11        instance.init(2, secretKeySpec);
12        return instance.doFinal(bArr2);
13    }
14 }

```

Như đoạn code trên sẽ so sánh giữa biến *str* và kết quả của hàm

```
sg.vantagepoint.a.a(b("8d127684cbc37c17616d806cf50473cc"), Base64.decode("5UJiFctbmgbdLXmpl12mkno8HT4Lv8dlat8FxR2G0c=", 0))
```

Sau khi hiểu cách thức hoạt động xong mình nghĩ ra được hai cách để bypass

Cách 1: Dùng *Java.use()* để thực hiện *return true*;

Đọc code đến đây là có thể dùng Frida để alert dòng chữ *Success* lên với đoạn code tí xâu này.

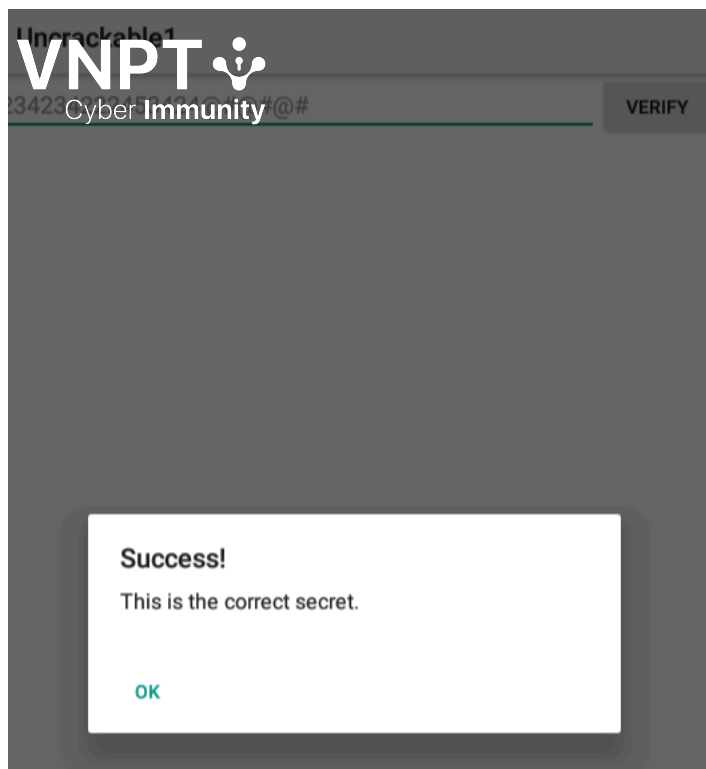
```

var check = Java.use("sg.vantagepoint.uncrackable1.a");
check.a.implementation = function() {
    console.log("vao ham check secret");
    return true;
};

```

Lúc này, bạn nhập bất kỳ chuỗi ký tự nào vào cũng sẽ alert *Success* lên





Cách 2: Là tìm ra chuỗi **secret string** luôn. Mình thường dùng cách này cần thu thập các chuỗi ký tự đã được mã hóa trong các ứng dụng mà mình gặp phải. Việc cần làm gọi vào hàm a() ở trên để trả về mảng byte, lúc này cần phải thêm một bước nữa là chuyển mảng **byte** thành **String**

```
// cách 2
var check = Java.use("sg.vantagepoint.a.a");
check.a.implementation = function(x,y) {
  console.log("vao ham decrypt");
  var bte = this.a(x,y);
  console.log("byte: " + bte);
  var result;
  for (var i = 0; i < bte.length; i++) {
    result += String.fromCharCode(bte[i]);
  }
  console.log("secret key: " + result);
  return bte;
};
```

Trong đó:

- Do hàm a() có hai arguments nên trong code js cần phải truy cập vào 2 x, y
- Gọi hàm a(x,y) với cú pháp **this.a(x,y)** để lấy giá trị ở kiểu dữ liệu byte

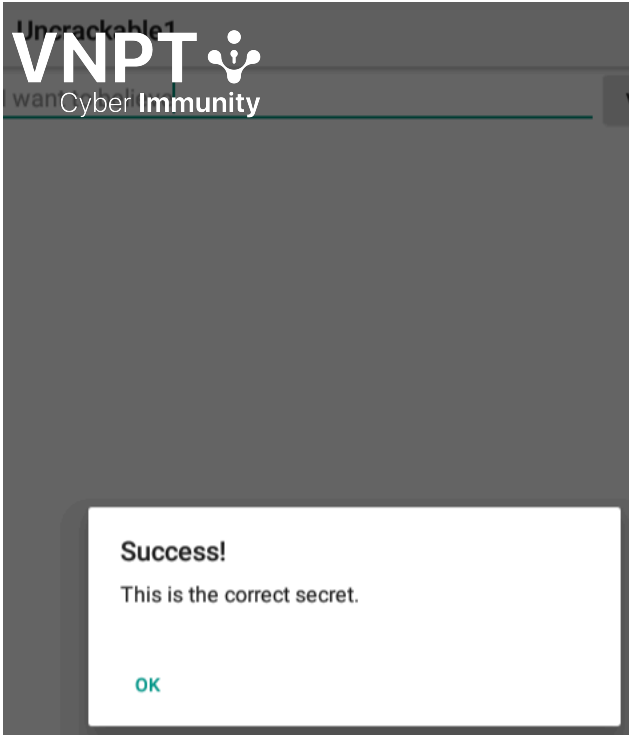
Kết quả: vậy secret string cần tìm là **I want to believe**

```
PS C:\Users\NoName\Desktop\blog_frida> frida -U --no-pause -l .\bypass_root.js -f owasp.mstg.uncrackable1

----
/ _ _ |   Frida 15.0.2 - A world-class dynamic instrumentation toolkit
| C _ |
> _ _ |   Commands:
/_/_/_|   help       -> Displays the help system
. . . .   object?    -> Display information about 'object'
. . . .   exit/quit  -> Exit
. . . .
. . . .   More info at https://frida.re/docs/home/
Spawned 'owasp.mstg.uncrackable1'. Resuming main thread!
[SM-G965N:owasp.mstg.uncrackable1]-> vao ham a
vao ham b
vao ham c
vao ham decrypt
byte: 73,32,119,97,110,116,32,116,111,32,98,101,108,105,101,118,101
secret key: undefinedI want to believe
```

Okay, nhấp vào thôi





Ngoài ra có thể tham khảo thêm Javascript API (<https://frida.re/docs/javascript-api>) thêm ở đây, với các hàm cơ bản như `Java.choose()`: Tìm instance của className bằng cách scan trong Java heap, nếu trùng khớp sẽ gọi đến `onMatch()`, ...

- `Java.choose(className, callbacks)`: enumerate live instances of the `className` class by scanning the Java heap, where `callbacks` is an object specifying:
 - `onMatch(instance)`: called with each live instance found with a ready-to-use `instance` just as if you would have called `Java.cast()` with a raw handle to this particular instance. This function may return the string `stop` to cancel the enumeration early.
 - `onComplete()`: called when all instances have been enumerated
- Bypass SSL pinning trong Flutter application

Những kiến thức trên chỉ là phần cơ bản, hy vọng có thể giúp những bạn mới có thể làm quen với hooking với Frida.
Còn thực tế những mobile app mình gặp ở môi trường thực tế sẽ khó khăn hơn nhiều do lượng code lớn, hàm check root, mã hóa, ... phức tạp hơn nhiều (mobile app của các ngân hàng)
Phần lớn các mobile app hiện nay dùng các framework như Flutter hoặc React Native, có thể vừa build android và iOS applications với cùng một codebase. Đặc biệt là Flutter này không tuân theo cấu trúc proxy của thiết bị được cài bởi pentester mà sẽ sử dụng certificate của lập trình viên cài đặt. Do đó, không thể bắt được request theo cách thông thường được.
Hiện nay có tool reFlutter (<https://github.com/Impact-1/reFlutter>) cho phép bypass SSL Pinning và ý tưởng của tool này khá đơn giản như sau: vì Flutter app thường sử dụng binary (libflutter.so bao gồm các Flutter engine) đã được compiled (ahead-of-time – AOT), vậy nên tool này chỉ cần sử dụng bản patch (<https://github.com/humblepoti/flutter-modified>) của Flutter library đã được compiled sẵn, sau đó re-packing app là ok. Nhưng sẽ có vấn đề xảy ra là nếu version Dart khác nhau sẽ không thể hoạt động được.
Đây là lý do chính khiến mình viết blog này.
Ở phần này thì cần thêm tool rev như: Ghidra hoặc ida
Hình dưới cho thấy thư viện được load vào ứng dụng

```
431 public void loadLibrary() {
432     if (loadLibraryCalled) {
433         C3526b.m3031f(TAG, "FlutterJNI.loadLibrary called more than once");
434     }
435     System.loadLibrary("flutter");
436     loadLibraryCalled = true;
437 }
```

Cấu hình proxy ở proxy-droid cho qua Burp Suite rồi mở app lên bạn sẽ được thông báo như sau ở tab Dashboard: **The client failed to negotiate a TLS connection to test.com:443: Remote host terminated the handshake**
Lúc này tất nhiên sẽ không thể bắt được http traffic

Event log			
Filter Critical Error Info Debug			
Search...			
Time	Type	Source	Message
16:45:05 27 Apr 2022	Error	Proxy	[18] The client failed to negotiate a TLS connection to [REDACTED] Remote host termi...

Đầu tiên cũng cực kỳ quan trọng là cần kiểm tra kiến trúc trên thiết bị để đảm bảo rằng load đúng thư viện bằng command sau:

```
adb shell getprop ro.product.cpu.abi
```



```
PS D:\Program Files\Nox\bin> .\adb.exe shell getprop ro.product.cpu.abi
```



Bước tiếp theo: import libflutter.so vào vào Ghidra, thời gian import khá lâu, thôi để đó, bật logcat lên xem thử như nào
 Nhận thấy rằng verify_perfect không phải là file handshake.cc (<https://github.com/google/boringssl/blob/master/ssl/handshake.cc>)

```
04-27 14:59:43.623 30640 30676 E flutter : [ERROR:flutter/lib/ui/ui_dart_state.cc(209)] Unhandled Exception: HandshakeExcepti
04-27 14:59:43.623 30640 30676 E flutter :       BLOCK_TYPE_IS_NOT_01(padding.c:108)
04-27 14:59:43.623 30640 30676 E flutter :       PADDING_CHECK_FAILED(rsa_impl.c:673)
04-27 14:59:43.623 30640 30676 E flutter :       public key routines(a_verify.c:104)
04-27 14:59:43.623 30640 30676 E flutter :       CERTIFICATE_VERIFY_FAILED: certificate signature failure(handshake.cc:359))
04-27 14:59:43.623 30640 30676 E flutter : #0     _SecureFilterImpl._handshake (dart:io-patch/secure_socket_patch.dart:101)
04-27 14:59:43.623 30640 30676 E flutter : #1     _SecureFilterImpl._handshake (dart:io-patch/secure_socket_patch.dart:143)
04-27 14:59:43.623 30640 30676 E flutter : #2     _RawSecureSocket._secureHandshake (dart:io/secure_socket.dart:794)
04-27 14:59:43.623 30640 30676 E flutter : #3     _RawSecureSocket._tryFilter (dart:io/secure_socket.dart:924)
04-27 14:59:43.623 30640 30676 E flutter : <asynchronous suspension>
04-27 14:59:43.623 30640 30676 E flutter : #4     _RawSecureSocket._secureHandshake (dart:io/secure_socket.dart:802)
```

Sau khi đầu đọc blog của anh [horangi](#) thì nhận thấy rằng phần thực thi tại hàm `session_verify_cert_chain` và sẽ trả về kết quả `ssl_verify_ok` hoặc `ssl_verify_invalid` và lưu trong biến `ret`

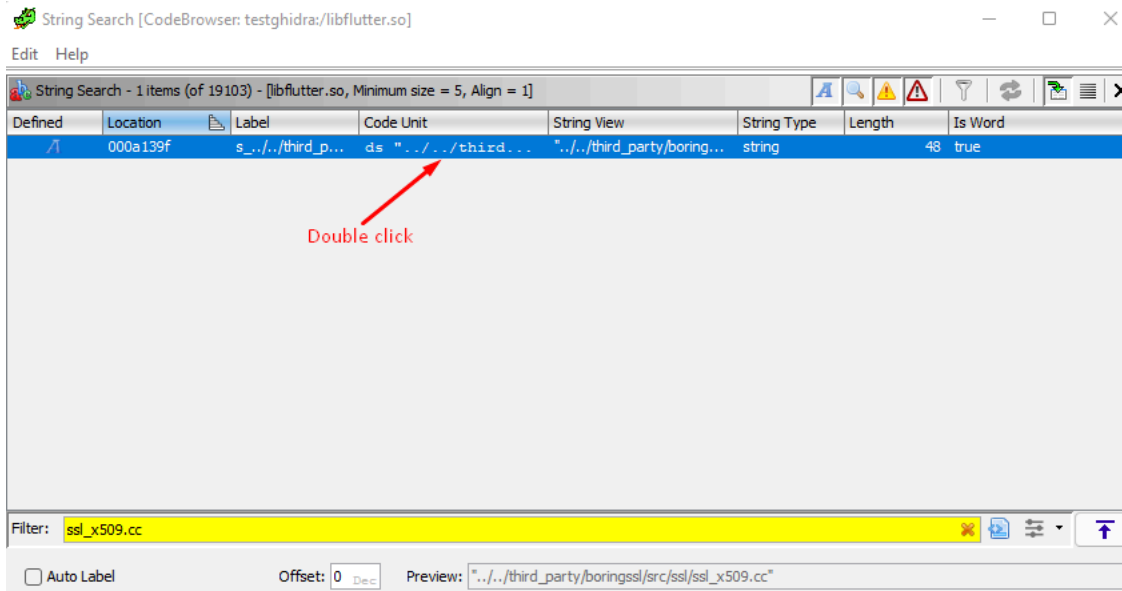
```
ret = ssl->ctx->x509_method->session_verify_cert_chain(
    hs->new_session.get(), hs, &alert)
? ssl_verify_ok
: ssl_verify_invalid;

if (ret == ssl_verify_invalid) {
    OPENSSL_PUT_ERROR(SSL, SSL_R_CERTIFICATE_VERIFY_FAILED);
    ssl_send_alert(ssl, SSL3_AL_FATAL, alert);
}
```

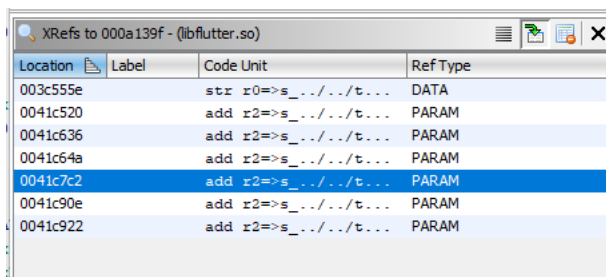
Định nghĩa của hàm `session_verify_cert_chain` nằm trong file `ssl_x509.cc` (https://github.com/google/boringssl/blob/master/ssl/ssl_x509.cc)

Okay, mục tiêu giờ phải thay đổi kết quả trả về của hàm trên thành true. Và cũng lưu ý rằng hàm trên có 3 arguments được truyền vào (sẽ giúp ta tìm được hàm chính xác khi decompile bằng Ghidra)

Chuyển qua Ghidra click Search -> For Strings... tìm `ssl_x509.cc` sẽ nhận được kết quả như hình bên dưới
 Double click vào kết quả tìm được, bạn sẽ thấy được các References (như trong hình có 7 chỗ gọi đến string này)



Double click vào `XREF[7]` sẽ hiển thị như hình dưới



Lúc này phải duyệt qua tất cả các Xrefs để tìm được function có 3 arguments (đã nói lưu ý ở trên) và return 0

```

1 |
2 | undefined function_0041c704(int param_1,int *param_2,undefined *param_3)
3 |
4 | {
5 |     undefined uVar1;
6 |     int iVar2;
7 |     char *pcVar3;
8 |     int iVar4;
9 |     undefined4 uVar5;
10 |    uint uVar6;
11 |    int *piVar7;
12 |    int iVar8;
13 |    undefined auStack168 [16];
14 |    int iStack152;
15 |    int iStack140;
16 |    int iStack72;
17 |    undefined auStack44 [8];
18 |
19 |    *param_3 = 0x50;
20 |    piVar7 = *(int **) (param_1 + 0x98);
21 |    if ((piVar7 == (int *)0x0) || (*piVar7 == 0)) {

```

```

73 |    uVar5 = 0;
74 | LAB_0041c7ca:
75 |    FUN_00407f24(auStack168);
76 |    return uVar5;
77 | }

```

Тим được đến đây thì có hai sự lựa chọn để giải quyết bài toán bypass SSL Pinning:
 Cách thứ nhất: patch lại app chính sửa 0x0 thành 0x1

Затем сохраните и выберите «Экспортировать программу», используйте **apktool.jar** для компиляции приложения, затем используйте **uber-apk-signer.jar** для подписи приложения.

Второй способ: используйте Фриду для перехвата, получите шестнадцатеричный код, затем используйте код для обхода

0041c704 74 46 4e 4e push { r4, r5, r6, r7, r8, r9, r10, r11, lr }

0 41 0a 0a sub sp, #0x8c

041c70e 10 70 00 00 mov r9, r0

041c710 d9 f8 98 70 movs r0, #0x50

041c714 00 2f 00 00 ldr r0, [r2, #0x0]

041c716 4d d0 beq LAB_0041c7b4

041c718 38 68 ldr r0, [r7, #0x0]

041c71a 00 28 cmp r0, #0x0

041c71c 4a d0 beq LAB_0041c7b4

041c71e 78 68 ldr r0, [r7, #0x4]

041c720 0d 46 mov r5, r1

041c722 0d f1 08 0a add.w r10, sp, #0x8

041c726 90 46 mov r8, r2

041c728 06 68 ldr r6, [r0, #0x0]

041c72a d1 e9 00 01 ldrd r0, r1, [r1, #0x0]

041c72e 09 69 ldr r1, [r1, #0x10]

041c730 40 6b ldr r0, [r0, #0x34]

041c732 d1 f8 2c b0 ldr.w r11, [r1, #0x2c]

041c736 80 21 movs r1, #0x80

041c738 01 90 str r0, [sp, #0x4]

041c73a c4 6c ldr r4, [r0, #0x4c]

041c73e 50 46 mov r0, r10

041c73e 24 f1 30 eb blx <EXTERNAL>: __aeabi_memclr8

041c742 bb f1 00 0f cmp.w r11, #0x0

041c746 18 bf it ne

041c748 5c 46 mov.ne r4, r11

041c74a 50 46 mov r0, r10

041c74c 21 46 mov r1, r4

041c74e 32 46 mov r2, r6

041c750 3b 46 mov r3, r7

041c752 eb f7 0f fc bl FUN_00407f74

041c756 78 b3 cbz r0, LAB_0041c7b8

041c758 2a 68 ldr r2, [r5, #0x0]

041c75a 0a f1 7c 00 add.w r0, r10, #0x7c

041c75e 00 21 movs r1, #0x0

041c760 cd f7 f2 ff bl FUN_003ea748

041c764 40 b3 cbz r0, LAB_0041c7b8

041c766 28 68 ldr r0, [r5, #0x0]

041c768 90 f8 58 00 ldrb.w r0, [r0, #0x58]

041c76c 00 07 lsls r0, r0, #0x1f

041c76e 29 48 ldr r0, [DAT_0041c814]

041c770 29 49 ldr r1, [DAT_0041c818]

2 undefined4 UndefinedFunction_0041c704 int param_1, int *param_2, undefined

3

4 {

5 undefined uVar1;

6 int iVar2;

7 char *pcVar3;

8 int iVar4;

9 undefined4 uVar5;

10 uint uVar6;

11 int *piVar7;

12 int iVar8;

13 undefined auStack168 [16];

14 int iStack152;

15 int iStack140;

16 int iStack72;

17 undefined auStack44 [8];

18

19 *param_3 = 0x50;

20 piVar7 = *(int *) (param_1 + 0x98);

21 if ((piVar7 == (int *)0x0) || (*piVar7 == 0)) {

22 return 0;

23 }

24 uVar5 = *(undefined4 *)piVar7[1];

25 iVar2 = *(int *) (*param_2 + 0x34);

26 iVar8 = *(int *) (* (int *) (param_2[1] + 0x10) + 0x2c);

27 iVar4 = *(int *) (iVar2 + 0x4c);

28 __aeabi_memclr8 (auStack168, 0x80);

29 if (iVar8 != 0) {

30 iVar4 = iVar8;

31 }

32 iVar4 = FUN_00407f74 (auStack168, iVar4, uVar5, piVar7);

33 if ((iVar4 == 0) || (iVar4 = FUN_003ea748 (auStack44, 0, *param_2), iVar4

34 LAB_0041c7b8:

35 FUN_003e5570 (0x10, 0xb, ".../third_party/boringssl/src/ssl/ssl_x509.c

36 }

37 else {

38 pcVar3 = "ssl_client";

39 if ((* (byte *) (*param_2 + 0x58) & 1) == 0) {

40 pcVar3 = "ssl_server";

41 }

42 iVar4 = FUN_00409368 (auStack168, pcVar3);

43 if (iVar4 == 0) goto LAB_0041c7b8;

44 uVar6 = *(uint *) (iStack152 + 8);

45 uVar6 = *(undefined4 *) (param_2[1] + 8);

0041c700

0041c710

0041c720

0041c730

0041c740

0041c750

0041c760

0041c770

0041c780

0041c790

0041c7a0

0041c7b0

0041c7c0

0041c7d0

0041c7e0

0041c7f0

0041c800

0041c810

0041c820

0041c830

0041c840

0041c850

0041c860

0041c870

0041c880

0041c890

0041c8a0

0041c8b0

0041c8c0

0041c8d0

0041c8e0

0041c8f0

0041c900

0041c910

0041c920

0041c930

0041c940

0041c950

0041c960

0041c970

0041c980

0041c990

0041c9a0

0041c9b0

0041c9c0

0041c9d0

0041c9e0

0041c9f0

0041ca00

0041ca10

0041ca20

0041ca30

0041ca40

0041ca50

0041ca60

0041ca70

0041ca80

0041ca90

0041caa0

0041cab0

0041cac0

0041cad0

0041cae0

0041caf0

0041cb00

0041cb10

0041cb20

0041cb30

0041cb40

0041cb50

0041cb60

0041cb70

0041cb80

0041cb90

0041cba0

0041cbb0

0041cbc0

0041cbd0

0041cbe0

0041cbf0

0041cc00

0041cc10

0041cc20

0041cc30

0041cc40

0041cc50

0041cc60

0041cc70

0041cc80

0041cc90

0041cca0

0041ccb0

0041ccc0

0041ccd0

0041cce0

0041ccf0

0041cd00

0041cd10

0041cd20

0041cd30

0041cd40

0041cd50

0041cd60

0041cd70

0041cd80

0041cd90

0041cda0

0041cdb0

0041cdc0

0041cde0

0041cdf0

0041ce00

0041ce10

0041ce20

0041ce30

0041ce40

0041ce50

0041ce60

0041ce70

0041ce80

0041ce90

0041cea0

0041ceb0

0041cec0

0041ced0

0041cee0

0041cef0

0041cf00

0041cf10

0041cf20

0041cf30

0041cf40

0041cf50

0041cf60

0041cf70

0041cf80

0041cf90

0041cfa0

0041cfb0

0041cfc0

0041cfd0

0041cfe0

0041cff0

0041d000

0041d010

0041d020

0041d030

0041d040

0041d050

0041d060

0041d070

0041d080

0041d090

0041d0a0

0041d0b0

0041d0c0

0041d0d0

0041d0e0

0041d0f0

0041d100

0041d110

0041d120

0041d130

0041d140

0041d150

0041d160

0041d170

0041d180

0041d190

0041d1a0

0041d1b0

0041d1c0

0041d1d0

0041d1e0

0041d1f0

0041d200

0041d210

0041d220

0041d230

0041d240

0041d250

0041d260

0041d270

0041d280

0041d290

0041d2a0

0041d2b0

0041d2c0

0041d2d0

0041d2e0

0041d2f0

0041d300

0041d310

0041d320

0041d330

0041d340

0041d350

0041d360

0041d370

0041d380

0041d390

0041d3a0

0041d3b0

0041d3c0

0041d3d0

0041d3e0

0041d3f0

0041d400

0041d410

0041d420

0041d430

0041d440

0041d450

0041d460

0041d470

0041d480

0041d490

0041d4a0

0041d4b0

0041d4c0

0041d4d0

0041d4e0

0041d4f0

0041d500

0041d510

0041d520

0041d530

0041d540

0041d550

0041d560

0041d570

0041d580

0041d590

0041d5a0

0041d5b0

0041d5c0

0041d5d0

0041d5e0

0041d5f0

0041d600

0041d610

0041d620

0041d630

0041d640

0041d650

0041d660

0041d670

0041d680

0041d690

0041d6a0

0041d6b0

0041d6c0

0041d6d0

0041d6e0

0041d6f0

0041d700

0041d710

0041d720

0041d730

0041d740

0041d750

0041d760

0041d770

0041d780

0041d790

0041d7a0

0041d7b0

0041d7c0

0041d7d0

0041d7e0

0041d7f0

0041d800

0041d810

0041d820

0041d830

0041d840

0041d850

0041d860

0041d870

0041d880

0041d890

0041d8a0

0041d8b0

0041d8c0

0041d8d0

0041d8e0

0041d8f0

0041d900

0041d910

0041d920

0041d930

0041d940

0041d950

0041d960

0041d970

0041d980

0041d990

0041d9a0

0041d9b0

0041d9c0

0041d9d0

0041d9e0

0041d9f0

0041da00

0041da10

0041da20

0041da30

0041da40

0041da50

0041da60

0041da70

0041da80

0041da90

0041daa0

0041dab0

0041dac0

0041dad0

0041dae0

0041daf0

0041db00

0041db10

0041db20

0041db30

0041db40

0041db50

0041db60

0041db70

0041db80

0041db90

0041dba0

0041dbb0

0041dbc0

0041dbd0

0041dbe0

0041dbf0

0041dc00

0041dc10

0041dc20

0041dc30

0041dc40

0041dc50

0041dc60

0041dc70

0041dc80

0041dc90

0041dca0

0041dcb0

0041dcd0

0041dce0

0041dcf0

0041dd00

0041dd10

0041dd20

0041dd30

0041dd40

0041dd50

0041dd60

0041dd70

0041dd80

0041dd90

0041dda0

0041ddb0

0041ddc0

0041ddd0

0041dde0

0041ddf0

0041de00

0041de10

0041de20

0041de30

0041de40

0041de50

0041de60

0041de70

0041de80

0041de90

0041dea0

0041deb0

0041dec0

0041ded0

0041dee0

0041def0

0041df00

0041df10

0041df20

0041df30

0041df40

0041df50

0041df60

0041df70

0041df80

0041df90

0041dfa0

0041dfb0

0041dfc0

0041dfd0

0041dfe0

0041dff0

0041e000

0041e010

0041e020

0041e030

0041e040

0041e050

0041e060

0041e070

0041e080

0041e090

0041e0a0

0041e0b0

0041e0c0

0041e0d0

0041e0e0

0041e0f0

0041e100

0041e110

0041e120

0041e130

0041e140

0041e150

0041e160

0041e170

0041e180

0041e190

0041e1a0

0041e1b0

0041e1c0

0041e1d0

0041e1e0

0041e1f0

0041e200

0041e210

0041e220

0041e230

0041e240

0041e250

0041e260

0041e270

0041e280

0041e290

0041e2a0

0041e2b0

0041e2c0

0041e2d0

0041e2e0

0041e2f0

0041e300

0041e310

0041e320

0041e330

0041e340

0041e350

0041e360

0041e370

0041e380

0041e390

0041e3a0

0041e3b0

0041e3c0

0041e3d0

0041e3e0

0041e3f0

0041e400

0041e410

0041e420

0041e430

0041e440

0041e450

0041e460

0041e470

0041e480

0041e490

0041e4a0

0041e4b0

0041e4c0

0041e4d0

0041e4e0

0041e4f0

0041e500

0041e510

0041e520

0041e530

0041e540

0041e550

0041e560

0041e570

0041e580

0041e590

0041e5a0

0041e5b0

0041e5c0

0041e5d0

0041e5e0

0041e5f0

0041e600

0041e610

0041e620

0041e630

0041e640

0041e650

0041e660

0041e670

0041e680

0041e690

0041e6a0

0041e6b0

0041e6c0

0041e6d0

0041e6e0

0041e6f0

0041e700

0041e710

0041e720

0041e730

0041e740

0041e750

0041e760

0041e770

0041e780

0041e790

0041e7a0

0041e7b0

0041e7c0

0041e7d0

0041e7e0

0041e7f0

0041e800

0041e810

0041e820

0041e830

0041e840

0041e850

0041e860

0041e870

0041e880

0041e890

0041e8a0

0041e8b0

0041e8c0

0041e8d0

0041e8e0

0041e8f0

0041e900

0041e910

0041e920

0041e930

0041e940

0041e950

0041e960

0041e970

0041e980

0041e990

0041e9a0

0041e9b0

0041e9c0

0041e9d0

0041e9e0

0041e9f0

0041ea00

0041ea10

0041ea20

0041ea30

0041ea40

0041ea50

0041ea60

0041ea70

0041ea80

0041ea90

0041eaa0

0041eab0

0041eac0

0041ead0

0041eae0

0041eaf0

0041eb00

0041eb10

0041eb20

0041eb30

0041eb40

0041eb50

0041eb60

0041eb70

0041eb80

0041eb90

0041eba0

0041ebb0

0041ebc0

0041ebd0

0041ebe0

0041ebf0

0041ec00

0041ec10

0041ec20

0041ec30

0041ec40

0041ec50

0041ec60

0041ec70

0041ec80

0041ec90

0041eca0

0041ecb0

0041ecc0

0041ecd0

0041ece0

0041ecf0

0041ed00

0041ed10

0041ed20

0041ed30

0041ed40

0041ed50

0041ed60

0041ed70

0041ed80

0041ed90

0041eda0

0041edb0

0041edc0

0041edd0

0041ede0

0041edf0

0041ee00

0041ee10

0041ee20

0041ee30

0041ee40

0041ee50

0041ee60

0041ee70

0041ee80

0041ee90

0041eea0

0041eeb0

0041eec0

0041eed0

0041eee0

0041eef0

0041ef00

0041ef10

0041ef20

0041ef30

0041ef40

0041ef50

0041ef60

0041ef70

0041ef80

0041ef90

0041efa0

0041efb0

0041efc0

0041efd0

0041efe0

0041eff0

0041f000

0041f010

0041f020

0041f030

0041f040

0041f050

0041f060

0041f070

0041f080

0041f090

0041f0a0

0041f0b0

0041f0c0

0041f0d0

0041f0e0

0041f0f0

0041f100

0041f110

0041f120

0041f130

0041f140

0041f150

0041f160

0041f170

0041f180

0041f190

0041f1a0

0041f1b0

0041f1c0

0041f1d0

0041f1e0

0041f1f0

0041f200

0041f210

0041f220

0041f230

0041f240

0041f250

0041f260

0041f270

0041f280

0041f290

0041f2a0

0041f2b0

0041f2c0

0041f2d0

0041f2e0

0041f2f0

0041f300

0041f310

0041f320

0041f330

0041f340

0041f350

0041f360

0041f370

0041f380

0041f390

0041f3a0

0041f3b0

0041f3c0

0041f3d0

0041f3e0

0041f3f0

0041f400

0041f410

0041f420

0041f430

0041f440

0041f450

0041f460

0041f470

0041f480

0041f490

0041f4a0

0041f4b0

0041f4c0

0041f4d0

0041f4e0

0041f4f0

0041f500

0041f510

0041f520

0041f530

0041f540

0041f550

0041f560

0041f570

0041f580

0041f590

0041f5a0

0041f5b0

0041f5c0

0041f5d0

0041f5e0

0041f5f0

0041f600

0041f610

0041f620

0041f630

0041f640

0041f650

0041f660

0041f670

0041f680

0041f690

0041f6a0

0041f6b0

0041f6c0

0041f6d0

0041f6e0

0041f6f0

0041f700

0041f710

0041f720

0041f730

0041f740

0041f750

0041f760

0041f770

0041f780

0041f790

0041f7a0

0041f7b0

0041f7c0

0041f7d0

0041f7e0

0041f7f0

0041f800

0041f810

0041f820

0041f830

0041f840

0041f850

0041f860

0041f870

0041f880

0041f890

0041f8a0

0041f8b0

0041f8c0

0041f8d0

0041f8e0

0041f8f0

0041f900

0041f910

0041f920

0041f930

0041f940

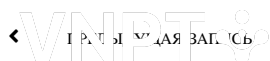
0041f950

0041f960

0041f970

0041f980

<



И Р. Т. И. У. Д. А. Я. З. А. Г. С. Ъ

Случайные мысли о паролях

(HTTPS://SEC.VNPT.VN/2022/04/TAN-MAN-VE-MAT-KHAU/)

БОЛЕЕ НОВЫЙ ПОСТ

Анализ предполагаемого вредоносного ПО АРТ, нацеленного на

Вьетнам

(HTTPS://SEC.VNPT.VN/2022/04/PHAN-TICH-MA-DOC-NGHI-NGO-TAN-CONG-APT-NHAM-VAO-VIET-NAM/)

ЦЕНТР ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

ИНФОРМАЦИОННО-ТЕХНОЛОГИЧЕСКАЯ КОМПАНИЯ VNPT (VNPT-IT)

Адрес головного офиса: 57 Huynh Thuc Khang - Dong Da - Ханой

Телефон: 0243.7265.171 / 0243.7265.276

 fb.com/vnptcyberimmunity(<http://fb.com/vnptcyberimmunity>)
 безопасность@vnpt.vn(<mailto:security@vnpt.vn>)

УСЛУГА

Мониторинг информационной безопасности (<https://sec.vnpt.vn/dich-vu-giai-phap-attt/#services>)Оценка информационной безопасности (<https://sec.vnpt.vn/dich-vu-giai-phap-attt/#services>)Обучение информационной безопасности (<https://sec.vnpt.vn/dich-vu-giai-phap-attt/#services>)Консультации по информационной безопасности (<https://sec.vnpt.vn/dich-vu-giai-phap-attt/#services>)

РЕШЕНИЕ

Защита DNS VNPT

(<https://sec.vnpt.vn/dich-vu-giai-phap-attt/#solutions>)

VNPT Смарт ИК

(<https://sec.vnpt.vn/dich-vu-giai-phap-attt/#solutions>)